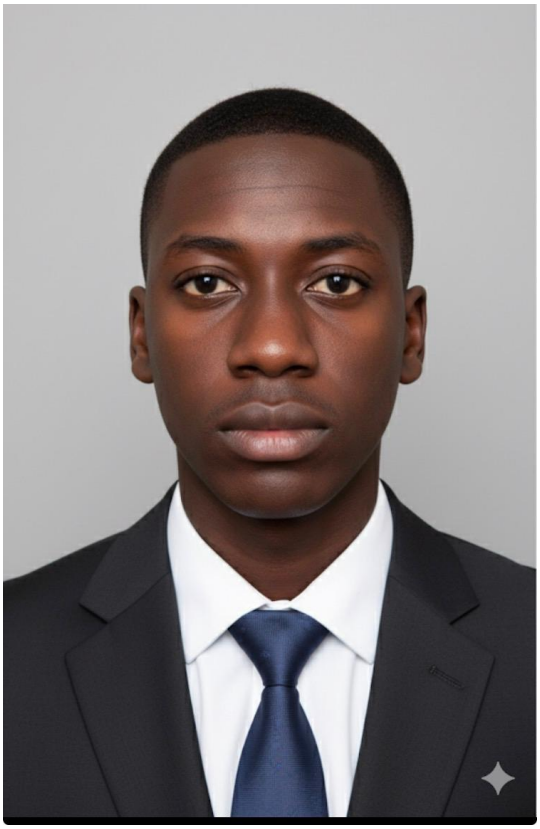


INSTITUTO TÉCNICO COMERCIAL DE LUANDA
COORDENAÇÃO DO CURSO DE INFORMÁTICA

Desenvolvimento de uma loja virtual para a venda de telemóveis, acessórios e consoles com serviço integrado de reparação de telemóveis

Luanda,2026

FICHA TÉCNICA



Nº 14 **Nome:** Lukenio kacupa José da Silva

Curso: Informática de Gestão

Turma: ATI

Sala: 16

Classe: 13ª

Luanda,2026

SUMÁRIO

Índice de figuras 1
Índice de tabelas 1
1. Introdução 2
1.1 Relevância do tema 3
2. Caso de estudo: a loja virtual LKN 3
3. Problema de pesquisa 4
4. Perguntas de pesquisa 4
5. Objetivos 5
5.1 Objetivo geral 5
5.2 Objetivos específicos 5
6. Estudo comparado das linguagens e tecnologias 5
6.1 Critérios de comparação 6
6.2 Linguagens do lado do servidor 6
6.3 Frameworks PHP 7
6.4 Sistemas de gestão de bases de dados 8
6.5 Tecnologias de front-end 9
6.6 Síntese do estudo comparado 9
7. Tecnologias utilizadas e justificação das escolhas 10
7.1 Visão geral da pilha tecnológica 10
7.2 PHP: a linguagem principal 11
7.3 Laravel: a framework 11
7.4 MySQL: a base de dados 12
7.5 HTML5, CSS3, JavaScript e Bootstrap: o front-end 13
7.6 Servidor web e ambiente de execução 14
7.7 Ferramentas de desenvolvimento 14
7.8 Pagamentos e integrações 14
7.9 Segurança da aplicação 14
7.10 Desempenho e escalabilidade 15
7.11 Síntese da justificação 15
8. Arquitetura do sistema 16
9. Conclusão 18
Referências bibliográficas 19

Índice de figuras

- Figura 1 – Arquitetura em camadas da loja LKN
- Figura 2 – Fluxo de um pedido de reparação
-

Índice de tabelas

- Tabela 1 – Comparação de linguagens do lado do servidor
- Tabela 2 – Comparação de abordagens em PHP
- Tabela 3 – Comparação de sistemas de gestão de bases de dados
- Tabela 4 – Comparação de abordagens de front-end
- Tabela 5 – Pilha tecnológica da loja LKN
- Tabela 6 – Principais entidades da base de dados da LKN
- Tabela 7 – Riscos de segurança e medidas adotadas

1. Introdução

O comércio eletrónico alterou de forma profunda a maneira como as pessoas compram bens de consumo, sobretudo produtos tecnológicos, cujas especificações são fáceis de comparar online e cuja procura é constante (Laudon & Traver, 2021). Telemóveis, acessórios e consolas de jogos reúnem estas características: têm catálogos extensos, preços que variam com frequência e exigem apoio depois da compra, como a reparação e a garantia.

A LKN é uma loja virtual que vende telemóveis, acessórios e consolas e que, além da venda, presta serviços de reparação de telemóveis. Funciona essencialmente como uma aplicação web: o cliente consulta o catálogo, compra e acompanha os seus pedidos através do navegador, enquanto a equipa da loja gere produtos, encomendas e reparações num painel de administração. Este trabalho estuda a LKN como caso de estudo e analisa as tecnologias mais adequadas para a construir, com destaque para a linguagem PHP.

1.1 Relevância do tema

O tema é relevante por três razões. Primeiro, a escolha das tecnologias condiciona o custo, a segurança e a facilidade de manutenção de qualquer aplicação web ao longo da sua vida (Sommerville, 2016). Segundo, uma loja que combina venda de produtos com prestação de serviços tem necessidades diferentes de uma loja comum, porque tem de gerir stock e também ordens de reparação com estados e prazos. Terceiro, o PHP continua a ser a linguagem de servidor mais usada na web: segundo a W3Techs (2026), é usada por cerca de 70,8% dos sites cuja linguagem de servidor é conhecida, o que garante comunidade, documentação e alojamento económico.

2. Caso de estudo: a loja virtual LKN

O estudo de caso é uma estratégia de investigação que analisa em profundidade uma situação concreta dentro do seu contexto real (Yin, 2015). Neste trabalho, o caso é a LKN, uma loja virtual de tecnologia com duas atividades complementares: o comércio de produtos e a prestação de serviços de reparação.

dos clientes compra a partir dele (Nielsen, 1993). A **manutenibilidade** exige código organizado, para que a loja possa crescer sem ser reescrita.

3. Problema de pesquisa

Uma boa formulação do problema delimita aquilo que a investigação pretende esclarecer (Gil, 2019). No caso da LKN, o problema nasce de uma dificuldade prática: a loja precisa de vender produtos e gerir reparações no mesmo sistema, mas a escolha errada de linguagem, base de dados ou framework pode tornar a aplicação lenta, insegura, cara de manter ou difícil de expandir.

Em lojas pequenas e médias, é comum que a gestão de encomendas e de reparações seja feita por meios dispersos, como mensagens, folhas de cálculo e papel. Isto provoca perda de informação, atrasos na comunicação com o cliente e falta de histórico. Ao mesmo tempo, existem muitas tecnologias disponíveis, e a decisão entre elas raramente é feita com critérios claros.

Enunciado do problema: que tecnologias web permitem construir, com segurança, bom desempenho e custo comportável, uma loja virtual que una a venda de telemóveis, acessórios e consolas à gestão de reparações, e por que razões o PHP e as tecnologias que o acompanham constituem uma escolha adequada para a LKN?

4. Perguntas de pesquisa

As perguntas de pesquisa transformam o problema em questões concretas a que o trabalho vai responder (Lakatos & Marconi, 2017). Neste estudo colocam-se cinco.

1. Que requisitos funcionais e não funcionais uma loja virtual com serviço de reparação, como a LKN, deve cumprir?
2. Que linguagens de programação do lado do servidor podem ser usadas e como se comparam em desempenho, custo, curva de aprendizagem e ecossistema?
3. Que sistema de gestão de base de dados e que framework são mais adequados para guardar e processar os dados da LKN?
4. Que tecnologias de front-end garantem uma interface simples e adaptada ao telemóvel?
5. Que medidas de segurança são necessárias para proteger clientes, pagamentos e dados pessoais?

5. Objetivos

5.1 Objetivo geral

Analisar e justificar as tecnologias a utilizar no desenvolvimento da loja virtual LKN, uma aplicação web para venda de telemóveis, acessórios e consolas e gestão de reparações de telemóveis.

5.2 Objetivos específicos

- Descrever o caso de estudo da LKN, os seus utilizadores e os requisitos do sistema.
- Comparar linguagens de programação, frameworks, bases de dados e tecnologias de front-end usadas em aplicações web.
- Selecionar a pilha tecnológica da LKN, com o PHP como linguagem principal, e explicar a razão de cada escolha.
- Propor uma arquitetura em camadas e o fluxo de um pedido de reparação.
- Identificar as principais medidas de segurança a aplicar na loja.
- Apresentar conclusões e recomendações para a implementação.

6. Estudo comparado das linguagens e tecnologias

Este capítulo compara as alternativas disponíveis em quatro camadas de uma aplicação web: a linguagem do servidor, a framework, a base de dados e o front-end. Escolher bem em cada camada é uma decisão de arquitetura, e as decisões de arquitetura são difíceis e caras de inverter depois de o sistema estar em produção (Sommerville, 2016).

6.1 Critérios de comparação

Cada tecnologia é avaliada segundo seis critérios. O **desempenho** mede a rapidez com que o sistema responde a muitos pedidos. A **segurança** considera os mecanismos que a tecnologia oferece para evitar vulnerabilidades. O **custo** inclui licenças e alojamento. A **curva de aprendizagem** indica quanto tempo uma equipa pequena demora a ser produtiva. O **ecossistema** refere bibliotecas, comunidade e documentação. A **adequação à LKN** pondera tudo isto face aos requisitos do capítulo 2. As avaliações das tabelas são qualitativas e foram elaboradas para este estudo a partir da documentação oficial de cada tecnologia e de Sommerville (2016).

6.2 Linguagens do lado do servidor

A linguagem do servidor trata dos pedidos, aplica as regras de negócio e fala com a base de dados. Cinco opções são comuns em lojas online.

PHP. É uma linguagem de script criada para a web, executada no servidor, que gera páginas HTML dinamicamente (PHP Group, 2026a). Tem funções nativas para sessões, formulários, ficheiros e bases de dados, e desde a versão 7 ganhou grande velocidade; a série 8 acrescentou compilação em tempo de execução (JIT), tipos mais rigorosos e novas construções de linguagem. É suportada por praticamente todos os serviços de alojamento, incluindo os mais baratos. Existem lojas prontas feitas em PHP, como WooCommerce, PrestaShop e OpenCart, o que mostra que a linguagem é madura para comércio eletrónico. A desvantagem histórica é a inconsistência de alguns nomes de funções antigas e a facilidade de escrever código desorganizado quando não se usa uma framework.

JavaScript com Node.js. Permite usar a mesma linguagem no servidor e no navegador. O Node.js assenta num modelo orientado a eventos com entrada e saída não bloqueantes, o que o torna forte em aplicações de tempo real, como chats e notificações (OpenJS Foundation, 2026). Em contrapartida, exige gerir o processo do servidor e o alojamento é menos universal e, por vezes, mais caro do que o do PHP.

Python com Django. O Django segue a filosofia de trazer tudo incluído: ORM, autenticação, painel de administração e proteções contra CSRF e injeção de SQL (Django Software Foundation, 2026). O Python é fácil de aprender e muito usado em ciência de dados. Para uma loja pequena, porém, o alojamento exige configuração própria e os serviços partilhados mais baratos raramente o suportam bem.

Java com Spring. É uma opção robusta, muito usada em grandes empresas e bancos, com excelente desempenho e tipagem forte. A curva de aprendizagem é mais exigente, o código é mais extenso e o consumo de recursos é maior, o que pesa numa loja de dimensão modesta (Sommerville, 2016).

C# com ASP.NET Core. É uma plataforma moderna, multiplataforma e rápida, com ferramentas de desenvolvimento muito completas (Microsoft, 2026). O ecossistema está bastante ligado ao universo Microsoft e o alojamento barato é menos comum do que para o PHP.

Tabela 1 – Comparação de linguagens do lado do servidor

Tecnologia	Desempenho	Segurança nativa	Custo de alojamento	Curva de aprendizagem	Ecossistema para lojas	Adequação à LKN
PHP	Bom	Boa, com funções para hash de palavras-passe e consultas preparadas	Muito baixo	Baixa	Muito amplo (WooCommerce, PrestaShop, OpenCart)	Muito alta
Node.js	Muito bom em tempo real	Depende de bibliotecas	Médio	Média	Amplo	Alta
Python/Django	Bom	Muito boa, com proteções incluídas	Médio	Baixa a média	Médio	Média a alta
Java/Spring	Muito bom	Muito boa	Médio a alto	Alta	Amplo, sobretudo empresarial	Média
C#/ASP.NET Core	Muito bom	Muito boa	Médio	Média	Médio	Média

6.3 Frameworks PHP

Mesmo escolhendo PHP, há uma segunda decisão: escrever o código à mão ou usar uma framework. Uma framework impõe uma estrutura, normalmente o padrão MVC (Modelo-Vista-Controlador), que separa os dados, a lógica e a apresentação (Gamma et al., 1994), e fornece soluções prontas para problemas recorrentes.

PHP puro dá liberdade total e não exige aprendizagem adicional, mas obriga a reinventar autenticação, validação e proteção contra ataques, com maior risco de erros. **Laravel** é a framework mais popular do ecossistema: inclui o ORM Eloquent, migrações de base de dados, o sistema de templates Blade, a ferramenta de linha de comandos Artisan e proteção contra CSRF por omissão (Laravel, 2026). **Symfony** é composta por componentes reutilizáveis e é muito usada em projetos empresariais, mas é mais complexa para iniciantes (Symfony, 2026). **CodeIgniter** é leve e rápida de aprender, com poucos requisitos, embora ofereça menos funcionalidades prontas (CodeIgniter Foundation, 2026).

Tabela 2 – Comparação de abordagens em PHP

Abordagem	Estrutura	Funcionalidades incluídas	Curva de aprendizagem	Manutenção a longo prazo	Adequação à LKN
PHP puro	Definida pelo programador	Mínimas	Baixa	Difícil	Baixa
Laravel	MVC	Muito completas (ORM, migrações, autenticação, filas)	Média	Muito boa	Muito alta
Symfony	MVC por componentes	Completas, configuráveis	Alta	Muito boa	Alta
CodeIgniter	MVC leve	Básicas	Baixa	Boa	Média

6.4 Sistemas de gestão de bases de dados

Uma loja guarda dados fortemente relacionados: clientes, encomendas, linhas de encomenda, produtos e reparações. O modelo relacional foi desenhado para este tipo de dados e garante consistência através de transações (Elmasri & Navathe, 2016).

MySQL é um SGBD relacional de código aberto, com o motor InnoDB que suporta transações e chaves estrangeiras (Oracle, 2026). Está disponível em quase todos os serviços de alojamento e combina-se de forma natural com o PHP. **PostgreSQL** é também relacional e de código aberto, conhecido pelo cumprimento rigoroso do padrão SQL e por recursos avançados, como tipos JSON poderosos (PostgreSQL Global Development Group, 2026); é excelente, mas a administração é um pouco mais exigente. **SQLite** guarda tudo num único ficheiro e não precisa de servidor, o que é ótimo para protótipos, mas a sua própria documentação desaconselha-o quando há muitos acessos de escrita simultâneos (SQLite, 2026). **MongoDB** guarda documentos flexíveis em vez de tabelas, o que ajuda quando o formato dos dados muda muito, mas para encomendas e stock as relações e transações do modelo relacional são mais diretas (MongoDB, 2026).

Tabela 3 – Comparação de sistemas de gestão de bases de dados

SGBD	Modelo	Transações e integridade	Disponibilidade no alojamento	Integração com PHP	Adequação à LKN
MySQL	Relacional	Sim (InnoDB)	Muito ampla	Excelente	Muito alta
PostgreSQL	Relacional	Sim	Ampla	Muito boa	Alta
SQLite	Relacional em ficheiro	Sim, com limites de concorrência	Não precisa de servidor	Boa	Baixa para produção
MongoDB	Documentos	Sim, com mais configuração	Média	Boa	Média

6.5 Tecnologias de front-end

O front-end é o que o cliente vê. A base é sempre HTML para a estrutura, CSS para o aspeto e JavaScript para a interatividade (Mozilla, 2026). Sobre ela podem usar-se bibliotecas e frameworks.

O **Bootstrap** fornece uma grelha responsiva e componentes prontos, como menus, cartões e formulários, que se adaptam a ecrãs pequenos (Bootstrap Team, 2026). O **React** permite construir interfaces complexas com componentes reutilizáveis, mas normalmente exige ferramentas de compilação e cuidados extra para que os motores de pesquisa leiam bem o conteúdo (Meta Platforms, 2026). Para uma loja, que precisa de aparecer nos resultados de pesquisa, páginas geradas no servidor em HTML são uma vantagem.

Tabela 4 – Comparação de abordagens de front-end

Abordagem	Responsividade	Complexidade de configuração	Visibilidade em motores de pesquisa	Adequação à LKN
HTML, CSS e JavaScript puros	Manual	Muito baixa	Muito boa	Média
HTML, CSS e JavaScript com Bootstrap	Muito boa, com grelha incluída	Baixa	Muito boa	Muito alta
React	Depende do desenho	Média a alta	Boa, com renderização no servidor	Média

6.6 Síntese do estudo comparado

A comparação mostra que não existe uma tecnologia melhor em termos absolutos: existe a mais adequada ao contexto. Para a LKN, uma loja de dimensão pequena a média, com orçamento limitado, equipa reduzida e necessidade de aparecer em pesquisas, a combinação que melhor equilibra os seis critérios é PHP com a framework Laravel, MySQL como base de dados e HTML, CSS e JavaScript com Bootstrap no front-end. O capítulo seguinte explica estas escolhas em detalhe.

7. Tecnologias utilizadas e justificação das escolhas

Este capítulo apresenta a pilha tecnológica proposta para a LKN e, sobretudo, explica porque foi escolhida cada peça. Em cada secção, a parte **Porquê?** liga a decisão aos requisitos do capítulo 2 e aos resultados do estudo comparado do capítulo 7.

7.1 Visão geral da pilha tecnológica

A pilha escolhida segue um modelo clássico e comprovado: um navegador pede páginas a um servidor web, que executa o código PHP, que por sua vez consulta uma base de dados MySQL. A Tabela 8 resume as tecnologias e o papel de cada uma.

Tabela 5 – Pilha tecnológica da loja LKN

Camada	Tecnologia	Função na LKN	Razão principal da escolha
Linguagem do servidor	PHP (8.4 ou 8.5)	Lógica de negócio, sessões, formulários	Suporte universal no alojamento e baixo custo
Framework	Laravel	Estrutura MVC, rotas, autenticação, validação	Rapidez de desenvolvimento e segurança por omissão
Base de dados	MySQL	Produtos, clientes, encomendas, reparações	Modelo relacional, transações e integração com PHP
Front-end	HTML5, CSS3, JavaScript	Páginas, estilo e interatividade	Padrões abertos e boa visibilidade em pesquisas
Biblioteca de interface	Bootstrap	Layout responsivo e componentes	Adaptação a ecrãs de telemóvel
Servidor web	Apache ou Nginx	Receber pedidos HTTP e entregar páginas	Maturidade e compatibilidade com PHP
Segurança de transporte	HTTPS (TLS)	Proteger dados em trânsito	Obrigatório para login e pagamentos
Gestão de dependências	Composer	Instalar bibliotecas PHP	Padrão do ecossistema PHP

7.2 PHP: a linguagem principal

O que é. O PHP é uma linguagem de programação de uso geral, concebida para o desenvolvimento web, que se integra diretamente com HTML e é executada no servidor (PHP Group, 2026a). Quando um cliente abre a página de um telemóvel na LKN, o servidor corre um script PHP que lê os dados do produto na base de dados, monta o HTML e envia-o ao navegador.

Versão. A escolha deve recair sobre uma versão que ainda receba correções de segurança. De acordo com a página oficial de versões suportadas, as séries 8.4 e 8.5 estão atualmente dentro do período de suporte, e cada série recebe dois anos de correções gerais mais dois de correções críticas de segurança (PHP Group, 2026b). Usar versões fora de suporte expõe a loja a falhas já conhecidas.

Funcionalidades relevantes para a LKN. O PHP oferece extensões nativas para trabalhar com bases de dados através do PDO, que permite consultas preparadas e, assim, protege contra injeção de SQL. Fornece as funções `password_hash` e `password_verify` para guardar palavras-passe de forma segura, gestão de sessões para manter o cliente autenticado, e o OPcache, que guarda o código compilado em memória e acelera as respostas. A partir do PHP 7 e, sobretudo, do PHP 8, a linguagem tem tipos declarados, enumerações e expressões mais claras, o que reduz erros e melhora a leitura do código (PHP Group, 2026a).

Porquê? Há quatro razões. A primeira é o **custo**: o PHP corre em praticamente qualquer alojamento, o que permite que uma loja pequena comece com um investimento reduzido. A segunda é a **maturidade**: segundo a W3Techs (2026), o PHP é usado por cerca de 70,8% dos sites cuja linguagem de servidor se conhece, pelo que há muita documentação, muitos programadores e muitas soluções para problemas de comércio eletrónico. A terceira é a **aprendizagem**: a curva é suave, o que importa num contexto escolar e para equipas pequenas. A quarta é o **ecossistema de lojas**: plataformas como WooCommerce, PrestaShop e OpenCart provam que a linguagem serve para vender online, e a LKN pode aprender com as suas soluções de catálogo e carrinho.

Boas práticas adotadas. O código segue a norma PSR-12 da PHP-FIG, que define o estilo de escrita e torna o código mais uniforme (PHP-FIG, 2026). As dependências são geridas com o Composer, que instala e atualiza bibliotecas de forma controlada (Composer, 2026).

7.3 Laravel: a framework

O que é. O Laravel é uma framework PHP que organiza a aplicação segundo o padrão MVC e oferece, já prontos, rotas, autenticação, validação de formulários, envio de correio eletrónico, filas de tarefas e uma camada de acesso a dados (Laravel, 2026). O padrão MVC separa o **Modelo**, que representa os dados e as regras, a **Vista**, que mostra a informação ao utilizador, e o **Controlador**, que recebe os pedidos e coordena as duas partes (Gamma et al., 1994).

Componentes usados.

- **Rotas:** definem que endereço chama que função, por exemplo /produtos para o catálogo e /reparacoes/nova para um novo pedido de reparação.
- **Eloquent ORM:** liga cada tabela a uma classe PHP, por exemplo Produto, Encomenda e Reparacao, de modo que o programador trabalha com objetos em vez de escrever SQL à mão.
- **Migrações:** ficheiros que descrevem a estrutura da base de dados e permitem recriá-la em qualquer máquina, o que facilita o trabalho em equipa.
- **Blade:** sistema de templates que gera HTML e escapa por omissão o conteúdo inserido, ajudando a evitar ataques XSS.
- **Middleware:** camadas que filtram pedidos, por exemplo para impedir que um cliente aceda à área de administração.
- **Validação:** regras declarativas para verificar dados, como email válido ou preço numérico.
- **Artisan:** ferramenta de linha de comandos que gera código e executa tarefas repetitivas.

Porquê? O Laravel reduz o tempo de desenvolvimento ao fornecer funcionalidades que, em PHP puro, teriam de ser escritas e testadas de raiz. Traz proteção contra CSRF e uso de consultas parametrizadas por omissão (Laravel, 2026), o que melhora a segurança sem esforço adicional. A estrutura MVC torna o projeto mais fácil de manter e de dividir entre vários elementos de um grupo. Em comparação com o Symfony, é mais acessível a iniciantes, e face ao CodeIgniter oferece mais funcionalidades incluídas, como o sistema de filas, útil para enviar notificações de estado da reparação (ver Tabela 5).

7.4 MySQL: a base de dados

O que é. O MySQL é um sistema de gestão de bases de dados relacional que organiza a informação em tabelas ligadas por chaves, consultadas com a linguagem SQL (Oracle, 2026). O motor InnoDB suporta transações, isto é, grupos de operações que ou se completam todas ou não se completam nenhuma, e chaves estrangeiras, que garantem que, por exemplo, uma encomenda nunca aponta para um cliente inexistente (Elmasri & Navathe, 2016).

Modelo de dados. A Tabela 9 apresenta as principais entidades da LKN.

Tabela 6 – Principais entidades da base de dados da LKN

Entidade	Principais atributos	Relações
Utilizador	Nome, email, palavra-passe cifrada, perfil	Tem muitas encomendas e muitas reparações
Categoria	Nome, descrição	Tem muitos produtos
Produto	Nome, descrição, preço, stock, marca	Pertence a uma categoria
Encomenda	Data, estado, total, morada de entrega	Pertence a um utilizador; tem muitos itens
Item de encomenda	Quantidade, preço na data da compra	Liga encomenda e produto
Pagamento	Método, estado, referência	Pertence a uma encomenda
Reparação	Aparelho, avaria, estimativa, estado, datas	Pertence a um utilizador; tem histórico
Histórico de reparação	Estado, data, nota técnica	Pertence a uma reparação

Porquê? Os dados de uma loja são naturalmente relacionais: um cliente faz várias encomendas, cada encomenda tem vários itens e cada item refere um produto. O modelo relacional representa isto com clareza e a integridade referencial evita registos inconsistentes. As transações são essenciais no momento da compra, quando é preciso descontar o stock e registar a encomenda como uma única operação. Por fim, o MySQL está disponível na maioria dos alojamentos e integra-se muito bem com o PHP e com o Laravel, o que reduz custos e problemas de configuração (ver Tabela 6).

7.5 HTML5, CSS3, JavaScript e Bootstrap: o front-end

O que são. O HTML5 define a estrutura das páginas, o CSS3 define o aspeto e o JavaScript acrescenta comportamento, como validar um formulário antes de o enviar ou atualizar o total do carrinho sem recarregar a página (Mozilla, 2026; Flanagan, 2020). O Bootstrap é uma biblioteca de CSS e JavaScript que traz uma grelha responsiva, botões, menus, cartões e formulários já estilizados (Bootstrap Team, 2026).

Aplicação na LKN. O catálogo é mostrado em cartões que se reorganizam consoante a largura do ecrã: várias colunas no computador, uma coluna no telemóvel. Os filtros de categoria e preço usam formulários HTML simples, melhorados com JavaScript para respostas mais rápidas. O formulário de pedido de reparação valida no navegador campos como marca, modelo e descrição da avaria, e repete a validação no servidor, porque a validação do lado do cliente não é suficiente para garantir segurança.

Porquê? A loja precisa de ser usada confortavelmente em telemóvel, e o Bootstrap resolve a responsividade com pouco código. O HTML gerado no servidor é lido facilmente pelos motores de pesquisa, o que ajuda os produtos da LKN a serem encontrados (ver Tabela 7). Esta abordagem também é mais simples de aprender do que uma aplicação de página única, e o JavaScript pode ser acrescentado apenas onde traz benefício real.

7.6 Servidor web e ambiente de execução

O servidor web recebe os pedidos HTTP e entrega a resposta. O **Apache** e o **Nginx** são os dois mais usados e ambos funcionam bem com PHP (Apache Software Foundation, 2026; Nginx, 2026). No desenvolvimento, pode usar-se um pacote local como o XAMPP para ter servidor, PHP e MySQL numa só instalação. Em produção, o PHP corre habitualmente através do PHP-FPM, que gere vários processos para atender muitos clientes em simultâneo.

A ligação deve ser protegida com **HTTPS**, que cifra a comunicação entre o navegador e o servidor. Certificados gratuitos e renováveis automaticamente podem ser obtidos através da Let's Encrypt (Let's Encrypt, 2026). **Porquê?** Sem HTTPS, palavras-passe e dados pessoais viajam em texto simples e os navegadores marcam o site como inseguro, o que afasta os clientes.

7.7 Ferramentas de desenvolvimento

O **Git** regista o histórico de alterações do código e permite que várias pessoas trabalhem em paralelo, com a possibilidade de voltar atrás em caso de erro (Chacon & Straub, 2014). O **Composer** gere as bibliotecas PHP. Para testes automáticos usa-se o PHPUnit, integrado no Laravel, que verifica se funções como o cálculo do total da encomenda continuam corretas depois de cada alteração. Um editor como o Visual Studio Code ou o PhpStorm completa o conjunto. **Porquê?** Estas ferramentas aumentam a qualidade e a segurança do projeto: sem controlo de versões, um erro pode destruir semanas de trabalho, e sem testes, cada alteração arrisca estragar o que já funcionava (Sommerville, 2016).

7.8 Pagamentos e integrações

A loja precisa de aceitar pagamentos. A abordagem recomendada é usar uma plataforma de pagamentos externa, comunicando com ela através de uma API, ou seja, um conjunto de pedidos HTTP com respostas em formato JSON, segundo o estilo arquitetural REST (Fielding, 2000). Com esta solução, os dados do cartão são introduzidos na página do prestador e nunca passam pelos servidores da LKN, o que reduz muito as obrigações de segurança associadas ao padrão PCI DSS (PCI Security Standards Council, 2022). A loja pode ainda oferecer pagamento na entrega ou por referência, consoante os meios disponíveis no mercado onde atua. O envio de mensagens de estado ao cliente faz-se por correio eletrónico através de SMTP, usando as filas do Laravel para não atrasar a resposta ao utilizador.

7.9 Segurança da aplicação

A segurança não é uma peça isolada: atravessa todas as camadas. A lista OWASP Top 10 reúne os riscos mais frequentes em aplicações web e serve de guia para as medidas a aplicar (OWASP Foundation, 2021). A Tabela 10 relaciona esses riscos com as soluções escolhidas.

Tabela 7 – Riscos de segurança e medidas adotadas

Risco	O que pode acontecer	Medida na LKN
Injeção (SQL)	Um atacante altera consultas à base de dados	Consultas preparadas com PDO e Eloquent
Autenticação fraca	Roubo de contas de clientes	Hash de palavras-passe, limite de tentativas de login, sessões seguras
Cross-Site Scripting (XSS)	Código malicioso aparece nas páginas	Escape automático no Blade e validação de entradas
Cross-Site Request Forgery (CSRF)	Ações feitas em nome do cliente sem o saber	Tokens CSRF do Laravel em todos os formulários
Controlo de acesso quebrado	Cliente acede à área de administração	Middleware e políticas por perfil
Exposição de dados sensíveis	Fuga de dados pessoais	HTTPS, ficheiro de configuração fora da pasta pública, cópias de segurança
Componentes vulneráveis	Falhas em bibliotecas desatualizadas	Atualizações regulares e verificação com composer audit
Configuração incorreta	Mensagens de erro revelam detalhes internos	Modo de depuração desligado em produção

A proteção dos dados pessoais deve ainda respeitar a legislação aplicável, como o Regulamento Geral sobre a Proteção de Dados na União Europeia ou a Lei da Proteção de Dados Pessoais em Angola (Parlamento Europeu e Conselho da União Europeia, 2016; Assembleia Nacional de Angola, 2011), que exigem consentimento, finalidade clara e cuidado na conservação dos dados.

7.10 Desempenho e escalabilidade

Uma loja lenta perde clientes. Para manter bom desempenho, a LKN deve usar índices nas colunas mais consultadas da base de dados, paginação nos catálogos, imagens comprimidas e dimensionadas ao tamanho do ecrã, e cache para guardar resultados que mudam pouco, como a lista de categorias. O OPcache do PHP e o sistema de cache do Laravel ajudam nesse sentido (PHP Group, 2026a; Laravel, 2026). Se o tráfego crescer, a arquitetura em camadas permite separar o servidor da base de dados e acrescentar servidores de aplicação sem reescrever o código.

7.11 Síntese da justificação

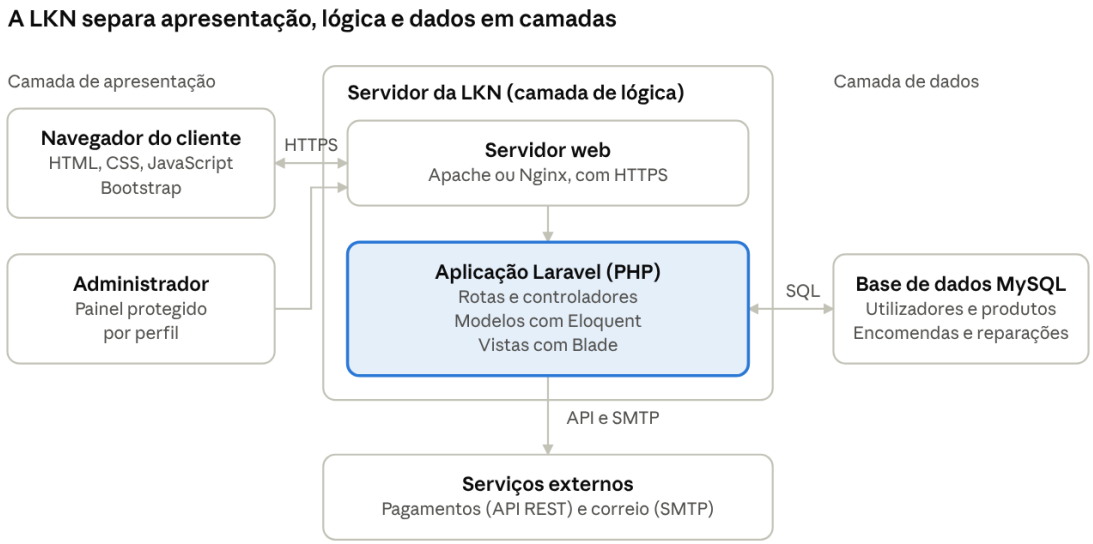
Em resumo, a pilha PHP, Laravel, MySQL e Bootstrap foi escolhida porque responde de forma equilibrada aos requisitos da LKN: é económica de alojar, rápida de desenvolver, segura se bem configurada, fácil de manter e capaz de crescer. Linguagens como Java ou C# oferecem grande robustez, mas implicam curvas de aprendizagem e custos que não se justificam para uma loja deste porte; o Node.js e o Python são excelentes, mas o alojamento e o ecossistema de lojas são menos favoráveis. A decisão tem também um limite: se a LKN vier a precisar de funcionalidades em tempo real muito intensas, como acompanhamento ao vivo de dezenas de reparações em simultâneo, poderá complementar a aplicação com um serviço em Node.js, sem abandonar a base já construída.

8. Arquitetura do sistema

8.1 Arquitetura em camadas

Uma arquitetura em camadas divide o sistema em partes com responsabilidades distintas, o que facilita a manutenção e permite alterar uma parte sem afetar as outras (Sommerville, 2016). A LKN usa três camadas. Na **camada de apresentação**, o navegador do cliente e o do administrador mostram as páginas e recolhem as ações. Na **camada de lógica**, o servidor web recebe os pedidos e entrega-os à aplicação Laravel, que aplica as regras de negócio. Na **camada de dados**, o MySQL guarda a informação de forma permanente. A aplicação comunica ainda com serviços externos, como a plataforma de pagamentos e o correio eletrónico.

Figura 1 – Arquitetura em camadas da loja LKN



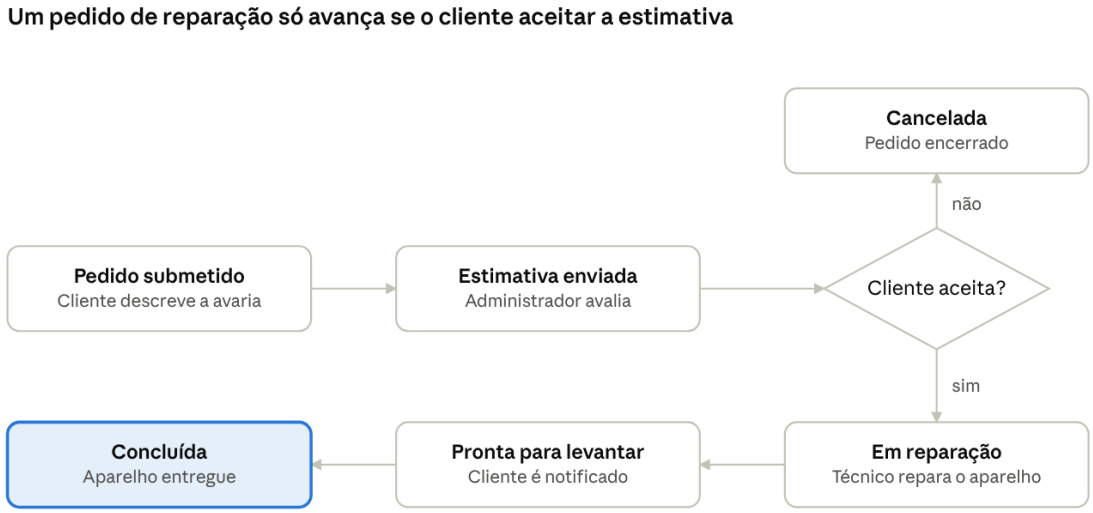
arquitetura em camadas · 3 camadas e serviços externos

Fonte: elaboração própria. Cliente e administrador usam o mesmo servidor, mas a aplicação mostra a cada um apenas o que o seu perfil permite.

8.2 Fluxo de um pedido de reparação

O serviço de reparação é o que distingue a LKN de uma loja comum, por isso o seu fluxo merece um desenho próprio. O cliente submete o pedido, descrevendo a avaria (RF05). O administrador avalia o caso e envia uma estimativa. Se o cliente recusar, o pedido fica cancelado; se aceitar, o aparelho entra em reparação e, depois, fica pronto para levantar e é entregue (RF06 e RF08).

Figura 2 – Fluxo de um pedido de reparação



fluxo de um pedido de reparação · 6 estados, 1 decisão

Fonte: elaboração própria. Cada mudança de estado é gravada na entidade Histórico de reparação (Tabela 9) e comunicada ao cliente por correio eletrónico, o que dá transparência ao processo e deixa um registo para consulta futura.

9. Conclusão

Este trabalho analisou a loja virtual LKN, que vende telemóveis, acessórios e consolas e presta serviços de reparação de telemóveis, com o objetivo de justificar as tecnologias mais adequadas para a construir. Respondendo às perguntas de pesquisa do capítulo 4, chega-se às seguintes conclusões.

1. **Requisitos.** A LKN precisa de um catálogo pesquisável, carrinho, encomendas, pedidos de reparação com acompanhamento e uma área de administração, com exigências de segurança, desempenho, usabilidade em telemóvel e facilidade de manutenção (Tabelas 1 e 2).
2. **Linguagem do servidor.** O PHP é a opção que melhor equilibra custo de alojamento, maturidade, facilidade de aprendizagem e existência de soluções para comércio eletrónico, embora Node.js, Python, Java e C# sejam alternativas válidas noutros contextos (Tabela 4).
3. **Framework e base de dados.** O Laravel acrescenta estrutura e segurança ao PHP, e o MySQL responde de forma natural aos dados relacionais e às transações de uma loja (Tabelas 5 e 6).
4. **Front-end.** HTML5, CSS3 e JavaScript com Bootstrap garantem páginas responsivas, legíveis pelos motores de pesquisa e simples de manter (Tabela 7).
5. **Segurança.** A proteção da loja depende de várias medidas combinadas, como consultas preparadas, hash de palavras-passe, tokens CSRF, controlo de acesso por perfil, HTTPS e atualizações regulares (Tabela 10).

O objetivo geral foi, assim, cumprido: a pilha PHP, Laravel, MySQL e Bootstrap, apoiada em Apache ou Nginx, Git e Composer, constitui uma escolha fundamentada para a LKN. A arquitetura em três camadas e o fluxo de reparações propostos (Figuras 1 e 2) mostram como as peças se encaixam.

Referências bibliográficas

Apache Software Foundation. (2026). *Apache HTTP Server documentation*. https://httpd.apache.org/docs/

Assembleia Nacional de Angola. (2011). *Lei n.º 22/11, de 17 de junho: Lei da Proteção de Dados Pessoais*. Diário da República.

Bootstrap Team. (2026). *Bootstrap documentation*. https://getbootstrap.com/docs/

Chacon, S., & Straub, B. (2014). *Pro Git* (2.ª ed.). Apress. https://git-scm.com/book

CodeIgniter Foundation. (2026). *CodeIgniter user guide*. https://codeigniter.com/user_guide/

Composer. (2026). *Composer documentation*. https://getcomposer.org/doc/

Django Software Foundation. (2026). *Django documentation*. https://docs.djangoproject.com/

Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of database systems* (7th ed.). Pearson.

Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* [Tese de doutoramento, University of California, Irvine].

Flanagan, D. (2020). *JavaScript: The definitive guide* (7th ed.). O'Reilly Media.

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). *Design patterns: Elements of reusable object-oriented software*. Addison-Wesley.

Gil, A. C. (2019). *Como elaborar projetos de pesquisa* (6.ª ed.). Atlas.

Lakatos, E. M., & Marconi, M. A. (2017). *Fundamentos de metodologia científica* (8.ª ed.). Atlas.

Laravel. (2026). *Laravel documentation*. https://laravel.com/docs

Parlamento Europeu e Conselho da União Europeia. (2016). *Regulamento (UE) 2016/679 (Regulamento Geral sobre a Proteção de Dados)*. Jornal Oficial da União Europeia, L 119.

https://www.pcisecuritystandards.org/

PHP Group. (2026a). *PHP manual*. https://www.php.net/manual/

PHP Group. (2026b). *Supported versions*. https://www.php.net/supported-versions.php

PHP-FIG. (2026). *PSR-12: Extended coding style guide*. https://www.php-fig.org/psr/psr-12/

PostgreSQL Global Development Group. (2026). *PostgreSQL documentation*.

Sommerville, I. (2016). *Software engineering* (10th ed.). Pearson.

SQLite. (2026). *Appropriate uses for SQLite*. https://www.sqlite.org/whentouse.html